FINAL PROJECT · DIGITAL DATABASE · 802244 · 2569

MOOD WEAR

ระบบฐานข้อมูลร้านค้าแฟชั่นพร้อม Back Office (Web) และแอปหน้าร้าน (Flutter)

ผู้นำเสนอ: รหัสนักศึกษา — ชื่อนักศึกษา · Repository: GitHub (mood-wear) · ไฟล์: 69db_รหัสนักศึกษา_02.pdf

ที่มาและแหล่งข้อมูลของระบบ

Mood Wear จำลองร้านค้าแฟชั่นจริงที่มีหน้าร้านให้ผู้ซื้อ (Flutter App) และหลังบ้านให้ผู้ดูแล (Web Back Office) — ทั้งสองฝั่งใช้ฐานข้อมูลชุดเดียวกันผ่าน REST API (FastAPI)

- แหล่งข้อมูล: **รวบรวมเอง + Web Scraping** รูป/รายการสินค้าแฟชั่นจริง (Samba, AMI Paris, Essentials, Human Made, Vivienne Westwood)
- ครอบคลุม 6 หมวด: Shoes · Upper · Bottom · Hat · Accessories · Set
- ชุดข้อมูลตัวอย่างผู้ใช้และการสั่งซื้อ สร้างความหลากหลายเพื่อสาธิตการค้นหา/สรุปยอด
- ข้อมูลส่วนบุคคลจำลองทั้งหมด — ไม่มีข้อมูลจริงของบุคคลใด

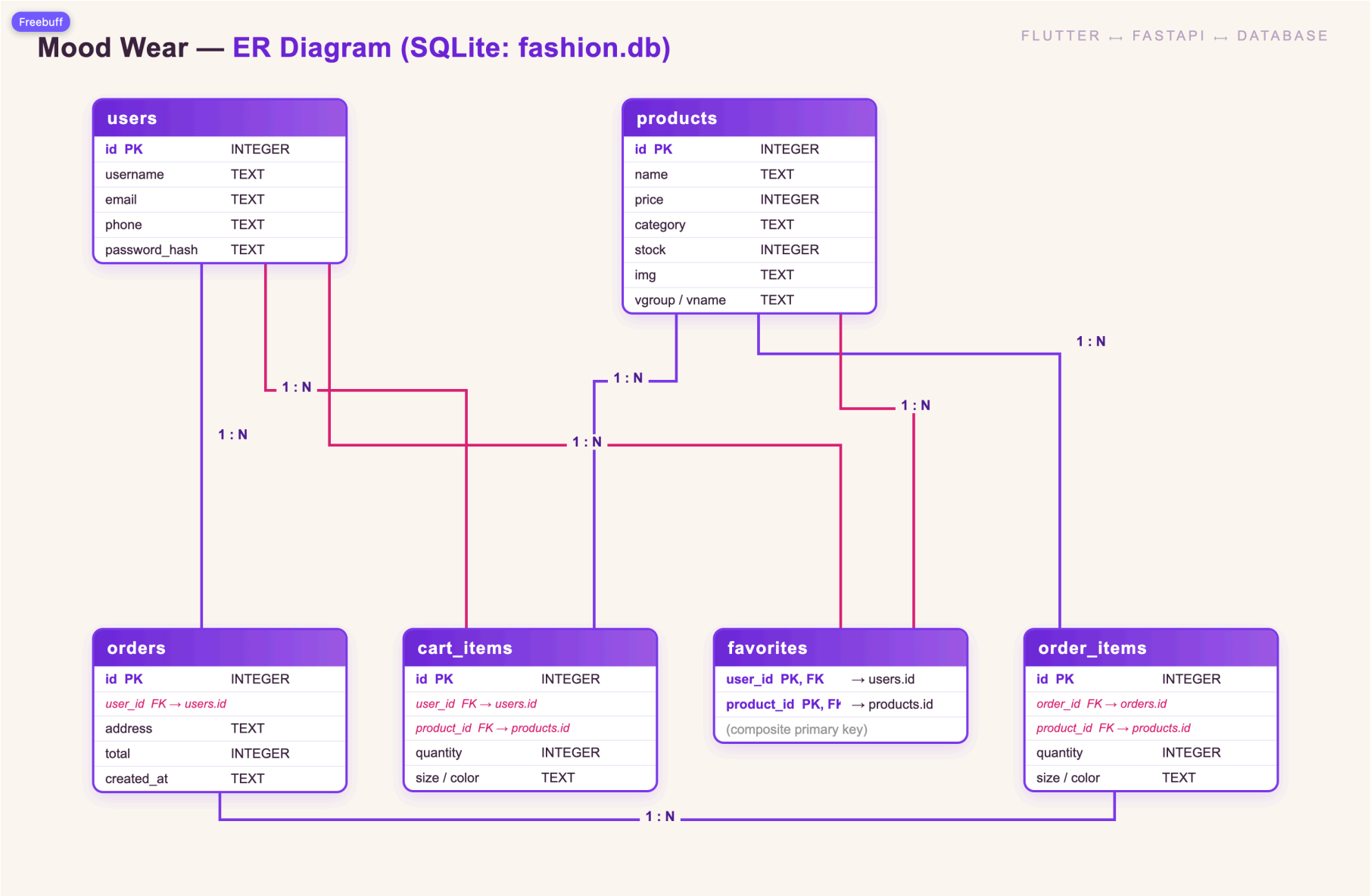
66 สินค้า	**51** ผู้ใช้	**58** คำสั่งซื้อ
111 รายการสั่งซื้อ	**52** ตะกร้า	**52** ถูกใจ

จำแนก Entity และ Attribute

วิเคราะห์ระบบงานได้ 5 Entity หลัก → แปลงเป็น 6 ตาราง (ORDER ITEM แยกเป็นตารางเชื่อม)

Entity	Attribute หลัก	บทบาทในระบบ
USER	id, username, email (unique), phone, password_hash	สมาชิกผู้ซื้อ — สั่งซื้อ / ตะกร้า / รายการถูกใจ
PRODUCT	id, name, price, category, stock, img, vgroup, vname, vorder	สินค้า + กลุ่ม variant (สินค้าชุดรูปเดียวกันหลายสี)
ORDER	id, user_id, address, total, created_at	ใบสั่งซื้อ 1 ครั้ง / ผู้ใช้ 1 คน
ORDER ITEM	id, order_id, product_id, quantity, size, color, unit_price	สินค้าแต่ละรายการในใบสั่งซื้อ
CART ITEM	id, user_id, product_id, quantity, size, color (UNIQUE)	สินค้าที่รอการสั่งซื้อ
FAVORITE	user_id + product_id (composite PK), created_at	สินค้าที่ผู้ใช้กดถูกใจ

ER Diagram ของระบบ



ER → Relation Schema (PK / FK)

ตาราง	Primary Key	Foreign Key	Constraint เพิ่มเติม
users	id	—	email UNIQUE
products	id	—	—
orders	id	user_id → users.id	created_at default
order_items	id	order_id → orders.id · product_id → products.id	unit_price เก็บราคา ณ ขณะสั่ง
cart_items	id	user_id · product_id	UNIQUE(user_id, product_id, size, color)
favorites	(user_id, product_id)	user_id · product_id	Composite PK — แปลงจาก M:N

เปิดบังคับ `PRAGMA foreign_keys = ON` — ทุก FK ถูกตรวจสอบทั้งตอน seed และตอน Back Office Insert/Update

การทำ Normalization

1NF ✔
ทุกตารางเก็บค่า atomic, มี PK ชัดเจน, ไม่มีกลุ่มข้อมูลซ้ำในช่องเดียว

2NF ✔
ตารางเดียวที่ใช้ composite key (favorites) ไม่มี attribute ที่พึ่งพาเพียงส่วนหนึ่งของ key → ไม่มี Partial Dependency

3NF ✔
ไม่มี Transitive Dependency — ชื่อ/ราคาสินค้าไม่ถูกเก็บซ้ำในตารางอื่น, unit_price เป็น “ราคาขณะสั่งซื้อ” ตามเงื่อนไขธุรกิจ

ผลลัพธ์: ข้อมูลถูกกระจายอย่างถูกต้อง แก้ไขตรงจุด และรวมสอบถามด้วย JOIN ได้ครบทุกความต้องการของระบบ

Data Dictionary (ตัวอย่างตารางหลัก)

products (66 แถว)	
id INTEGER PK	รหัสสินค้า
name TEXT	ชื่อสินค้า
price INTEGER	ราคา (บาท)
category TEXT	หมวดสินค้า
stock INTEGER	คงเหลือ
img, vgroup, vname, vorder	รูป + กลุ่ม variant + ลำดับ

order_items (111 แถว)	
id INTEGER PK	รหัสรายการ
order_id FK	ใบสั่งซื้อ
product_id FK	สินค้า
quantity, size, color	จำนวน/ไซส์/สี
unit_price	ราคาต่อชิ้น ณ ขณะสั่ง

users (51 แถว)	
id INTEGER PK	รหัสผู้ใช้
username, phone	ชื่อ / เบอร์โทร
email UNIQUE	อีเมล (ไม่ซ้ำ)
password_hash	รหัสผ่านเข้ารหัส

orders (58 แถว)	
id INTEGER PK	รหัสคำสั่งซื้อ
user_id FK	ผู้สั่งซื้อ
address, total	ที่อยู่ / ยอดรวม
created_at	เวลาสั่ง

cart_items (52) · favorites (52)	
cart_items: user_id FK, product_id FK	UNIQUE(user,product,size,color)
favorites: **(user_id, product_id)**	Composite PK + created_at

สถาปัตยกรรมระบบ และ REST API

แอปหน้าร้าน (Flutter Mobile/Web)

↓ HTTP/JSON

FastAPI Backend (Python) — ตรวจสอบข้อมูล + เปิด FK

↓

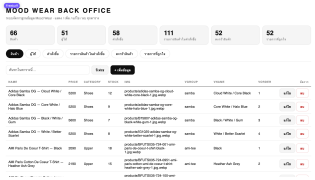
SQLite Database (fashion.db) · export เป็น MySQL ได้พร้อม

Back Office = ส่วนหน้าเดียวกันต่างหนึ่งเท่านั้น — ใช้ตารางเดียวกันจริง:
`/admin/t/{table}` CRUD · `/admin/stats` สรุปจำนวนแถว

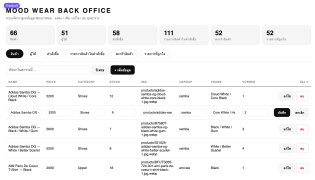
Method	Path	ใช้ทำ
GET	/products	รายการสินค้าหน้าร้าน
POST	/register · /login	สมัคร/เข้าสู่ระบบ (users)
POST	/cart · /checkout	ใส่ตะกร้า → สร้าง orders+order_items
GET/POST/DELETE	/favorites	รายการถูกใจ (composite key)
GET/POST/PUT/DELETE	/admin/t/{table} [/{id}]	Back Office CRUD ทุกตาราง
GET	/admin/stats	สถิติแถวทุกตาราง

Back Office — แสดง / เพิ่ม / แก้ไข / ลบ จริง

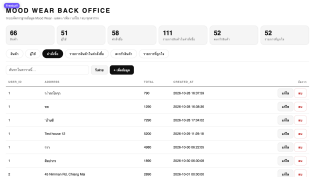
- **แสดง/ค้นหา:** แท็บ 6 ตาราง + ช่องค้นหา + การ์ดนับแถว
- **Insert:** ปุ่ม “+ เพิ่มข้อมูล” → ฟอร์มแถวใหม่ → บันทึกตรวจ FK
- **Update:** กด “แก้ไข” → แถวกลายเป็น input → บันทึก
- **Delete:** ปุ่ม “ลบ” พร้อมยืนยัน (รองรับ composite key ของ favorites)



ภาพรวมการ์ดแถว + แท็บ



โหมดแก้ไขในแถว



แท็บคำสั่งซื้อ

สาธิตเป็นวิดีโอ 60 วินาที (ไฟล์ 69db_รหัสนักศึกษา_03.mp4)

สรุป และวิธีติดตั้งระบบ

- มี **6 ตาราง** (เกินเกณฑ์ 5 Entity) และความสัมพันธ์ **6 แบบ** (เกิน 3 แบบ)
- ทุกตารางมีข้อมูล ≥ **50 แถว** จากข้อมูลรวบรวมเอง + Web Scraping
- PK/FK ถูกต้อง บังคับ ForeignKey จริง + ผ่าน Normalization 3NF
- Back Office แสดง/เพิ่ม/แก้ไข/ลบ ได้ทุกตาราง พร้อมวิดีโอสาธิต
- Source Code + README (อธิบายไฟล์ทุกไฟล์) พร้อมส่งขึ้น GitHub

ติดตั้ง

1. `pip install -r backend/requirements.txt`
2. `uvicorn main:app --port 8000` (seed ฐานข้อมูลเอง)

หรือ import `database_export_sqlite.sql` / `database_export_mysql.sql`

เข้าใช้งาน

Back Office: `http://127.0.0.1:8000/admin`
แอปหน้าร้าน: Flutter (run on device / Chrome)

ขอบคุณครับ 🙏 – ทีม Mood Wear